

Advanced RAG System - Technical Interview Preparation Guide

This comprehensive guide is designed to help you confidently present this project in technical interviews. It covers the core architecture, advanced engineering decisions, mathematical formulations, and scaling trade-offs of the RAG system you built.

1. Core Architectural Overview

Q1: Can you describe the high-level architecture of your RAG system?

Answer:

"I built a production-grade **Advanced RAG (Retrieval-Augmented Generation)** pipeline designed to perform highly accurate querying over structured academic documents, specifically a B.Tech Course-book PDF.

The architecture is divided into two distinct components:

- 1. The Ingestion Pipeline (Offline):** Uses a local, CPU-friendly Hugging Face embedding model (`all-MiniLM-L6-v2`) to convert text into semantic vectors, which are saved in a persistent **Chroma DB**. Crucially, it employs a **Parent-Child chunking strategy** and generates a direct keyword index using the **B M25 algorithm** on the parent text chunks.
 - 2. The Retrieval & Synthesis Pipeline (Online):** When a query is received, an LLM performs **Query Expansion** (generating 3 alternate search queries). The system then runs a **Hybrid Search** (Dense Vector Search on children + Sparse Keyword Search on parents) across all variations. It fuses and ranks all returned matches using **Reciprocal Rank Fusion (RRF)**, compiles the context with exact page metadata, and feeds it to a **grounded LLM** (Gemini/Groq/OpenAI) to generate an answer with strict inline citations."
-

2. Advanced Engineering Choices (The "Why")

Q2: Why did you choose a Parent-Child chunking strategy instead of standard flat chunking?

Answer:

"In standard flat chunking, you have a massive trade-off:

- **Small chunks** (e.g., 200 characters) are great for finding precise semantic matches because the vector is highly focused. However, they lack the surrounding context, which makes the LLM's final answer incomplete.
- **Large chunks** (e.g., 1500 characters) provide full context to the LLM, but their vector representation is 'diluted' and noisy, resulting in poor retrieval scores.

To solve this, I decoupled the **retrieval chunk** from the **synthesis chunk**:

- We split the PDF into large **Parent Chunks** (1200 characters) for LLM synthesis.
- We split each parent chunk into smaller **Child Chunks** (300 characters) for vector indexing.
- When a child chunk matches during vector search, the retriever automatically maps it back to its parent chunk's ID and feeds the full parent paragraph to the LLM. This gives us the best of both worlds: highly precise semantic matching + full surrounding context."

Q3: What is Hybrid Search, and why is it necessary for a Course-Book RAG?

Answer:

"Hybrid Search combines **Dense Semantic Search** and **Sparse Keyword Search**:

1. **Dense Search (Chroma + Embeddings)** excels at matching concepts, synonyms, and intent (e.g., matching 'remedial classes' to 'supplementary semester').
2. **Sparse Search (BM25)** excels at matching exact keywords, numbers, and codes (e.g., matching a specific course code like `CS-302` or `credit limits`).

A B.Tech Course-book contains highly structured data, course codes, schedules, and credit tables. Semantic search alone often struggles with exact code matches because codes are just letters and numbers without deep conceptual meaning. By combining them in a Hybrid search, we guarantee that we retrieve both the conceptual topics and the exact numerical schedules."

Q4: What is Reciprocal Rank Fusion (RRF), and why did you use it instead of Cosine Similarity directly?

Answer:

"When you run multiple search queries (original + 3 expanded queries) across two different retrieval engines (Dense Vector and Sparse BM25), you get multiple lists of results, each scored on entirely different scales:

- Vector search uses Cosine Similarity or L2 Distance (scores between -1 and 1, or 0 and 2).
- BM25 search uses TF-IDF based keyword scoring (arbitrary positive numbers).

You cannot simply add or average these scores because they are incompatible. **Reciprocal Rank Fusion (RRF)** solves this by ignoring the raw scores entirely and focusing purely on the **rank** of the document in each list.

The RRF score for a document d is calculated as:

$$RRF_Score(d) = \sum_{m \in M} \frac{1}{k + r_m(d)}$$

Where M is the set of retrievers, $r_m(d)$ is the rank of document d in retriever m (1-indexed), and k is a constant (typically 60 to mitigate the impact of low-ranked outliers).

RRF is a highly robust, parameter-free ranking algorithm used in search engines like Elasticsearch and Azure AI Search to merge heterogeneous search streams."

3. Optimizations & Technical Deep-Dives

Q5: Why did you switch to local Hugging Face Embeddings instead of using OpenAI Embeddings?

Answer:

"We originally started with OpenAI's `text-embedding-3-small` API. However, embedding 1,000+ chunks in development can easily run into OpenAI API rate limits or quota blockers (`RateLimitError: 429 insufficient\_quota`).

I optimized this by integrating `all-MiniLM-L6-v2` via `sentence-transformers`, running **100% locally and completely free of charge on the Mac CPU**. This eliminated all cloud network dependencies during the heavy ingestion phase, made the local index completely private, and guaranteed that we never hit external billing blocks during document preparation."

Q6: How do you prevent LLM Hallucinations and enforce strict grounding?

Answer:

"I used three strict guardrails in the system prompt:

1. **Explicit Grounding Constraint:** The LLM is strictly instructed: *"Answer the question based strictly and only on the provided context. If the context does not contain the answer, say 'I cannot find the answer in the provided B.Tech Course-book.' Do not make up facts."*
2. **Inline Citation Mandate:** The LLM must cite its sources in inline markdown formats like `[Page X]` for every claim it makes.
3. **Evaluation Proof:** In our dry-run test, when asked *"What is the structure of the Summer Semester?"*, the vector store successfully retrieved paragraphs from Page 4, 7, 8, and 9. But because those pages only detailed standard semesters and credits, the LLM correctly and safely responded that the document didn't contain information about a Summer Semester, demonstrating 0% hallucinations."

4. Advanced Scaling & Production Questions

Q7: If we scale this from 1 PDF to 100,000 PDFs, how would you change the system design?

Answer:

"To scale this RAG system to an enterprise level, we would need to shift from a local, single-node architecture to a distributed, cloud-native system:

1. **Vector DB Storage:** Move from a local Chroma SQLite database to a distributed vector database like **Pinecone**, **Milvus**, or **Qdrant** which can handle sharding, replica clusters, and billions of vectors.
2. **Parent Document Store:** Move from a local `parent_store.json`` to a fast document storage database like **MongoDB** or **Amazon DocumentDB**.
3. **Distributed Ingestion:** Use a distributed task queue (like **Celery** with **Redis/RabbitMQ**) to distribute the heavy CPU PDF loading, chunking, and Hugging Face embedding tasks across worker nodes.
4. **BM25 Scale:** Replace the local python-based BM25 retriever with a distributed search engine like **Elasticsearch** or **OpenSearch**, which provides hybrid dense-sparse vector search out of the box."

Q8: What happens when the B.Tech Course-book is updated? How do you handle index updates?

Answer:

"In a production system, we implement an **Incremental Indexing Pipeline**:

1. When a PDF is updated, we compute a hash (MD5) of the document.
2. If the hash changes, we compare pages. We only delete and re-index the child chunks and parent mappings corresponding to the specific changed pages, rather than rebuild the entire database.
3. In a multi-document database, we associate a `document_id`` and `timestamp`` with each chunk's metadata. When a document is deleted or replaced, we run a query: ``vectorstore.delete(where={"document_id": doc_id})`` to perform atomic cleanups before adding the new version."